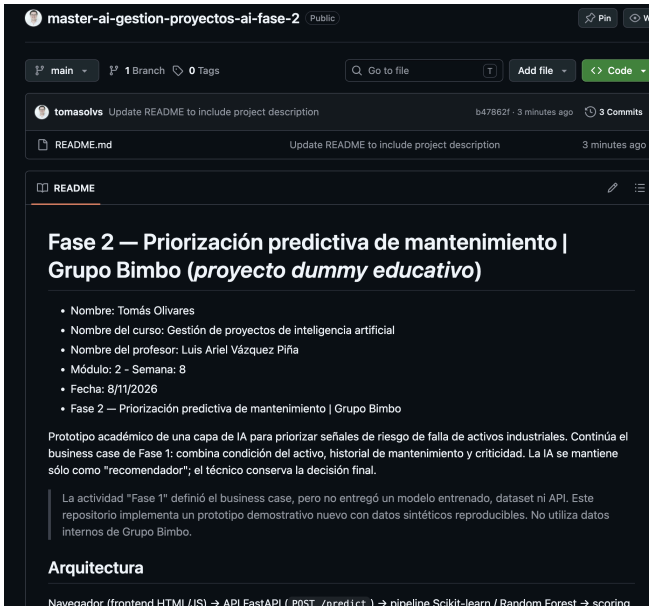
Nombre: Tomás Olivares Díaz de León	**Matrícula**: AL07183643
Nombre del curso: Gestión de proyectos de inteligencia artificial	**Nombre del profesor**: Luis Ariel Vázquez Piña
Módulo: 2 **Semana:** 8 **Fecha**: 8/11/2026	**Actividad**: Fase 2 — Priorización predictiva de mantenimiento

Fase 2 — Priorización predictiva de mantenimiento | Grupo Bimbo

Resumen ejecutivo



Prototipo académico de una capa de IA para priorizar señales de riesgo de falla de activos industriales. Continúa el business case de Fase 1: combina condición del activo, historial de mantenimiento y criticidad. La IA se mantiene sólo como "recomendador"; el técnico conserva la decisión final.

La actividad "Fase 1" definió el business case, pero no entregó un modelo entrenado, dataset ni API. Este repositorio implementa un prototipo demostrativo nuevo con datos sintéticos reproducibles. No utiliza datos internos de Grupo Bimbo.

Se incluyen **2 secciones en este documento**: **Validación y pruebas** y **Manual de despliegue en la nube**.

Repositorio en github: https://github.com/tomasolvs/master-ai-gestion-proyectos-ai-fase-2

Validación y pruebas

Evidencia de pruebas funcionales, casos extremos y resultados del prototipo

1. Alcance de validación

Se validó el contrato de la API, el estado del servicio, la generación de predicciones, la validación de entradas y el rechazo de valores fuera de rango. Las pruebas usan exclusivamente datos sintéticos. No constituyen validación operacional en plantas ni medición de reducción de fallas.

2. Modelo y datos

Algoritmo: RandomForestClassifier dentro de un pipeline con imputación por mediana. Dataset sintético reproducible de 3,000 registros; división estratificada 80/20; semilla 42. Variables: temperatura, vibración, presión, consumo energético, horas de operación, órdenes de trabajo en 90 días, días desde mantenimiento y criticidad del activo (1–5). Etiqueta: evento de falla dentro de un horizonte simulado, generado por una regla probabilística artificial.

Resultado de evaluación hold-out: ROC-AUC = 0.8091. Matriz de confusión [TN, FP; FN, TP] = [[452, 43], [54, 51]].

Clase de riesgo/falla (1): precision = 0.543; recall = 0.486; F1 = 0.513; support = 105.

Estas métricas sólo describen la separación sobre datos sintéticos generados por la misma lógica que define las etiquetas.

3. Pruebas automatizadas ejecutadas

Prueba	Resultado	Criterio
GET /health	APROBADA	HTTP 200 y status=ok
POST /predict válido	APROBADA	Devuelve score entre 0 y 1, nivel de riesgo y revisión humana
Lectura extrema dentro de rango	APROBADA	La API procesa lectura alta sin error
Criticidad fuera de rango (8)	APROBADA	HTTP 422
Vibración negativa	APROBADA	HTTP 422
Campo no permitido	APROBADA	HTTP 422

4. Casos extremos considerados

- Temperatura y vibración elevadas, muchas órdenes de trabajo, mantenimiento antiguo y criticidad 5: entrada extrema válida; la API responde y entrega un nivel de riesgo.
- Criticidad menor que 1 o mayor que 5: rechazada por validación de esquema.

- Vibración negativa: rechazada por validación de esquema.
- Campo adicional no declarado: rechazado para evitar aceptar silenciosamente atributos desconocidos.
- Campos numéricos sujetos a límites superiores para reducir entradas absurdas o potencialmente abusivas.

5. Pruebas manuales de integración

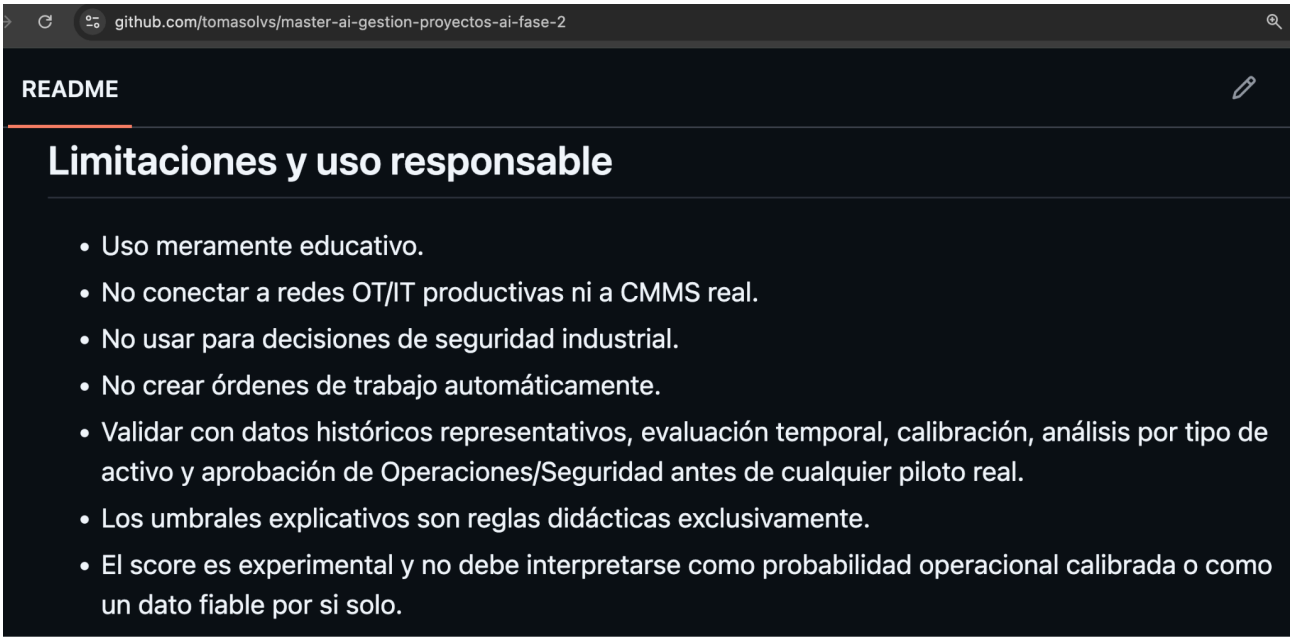
La interfaz web envía JSON mediante fetch a POST /predict. La API valida el payload con Pydantic, construye el vector de características, ejecuta predict_proba y responde con activo, probabilidad experimental, nivel de riesgo, prioridad, señales explicativas y recomendación de revisión humana. La documentación interactiva se publica en /docs.

Comando de prueba reproducible:

```
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d '{"asset_id":"MOTOR-001","temperature_c":92,"vibration_mm_s":8.2,"pressure_bar":6.5,"energy_kw":48,"operating_hours":18000,"work_orders_90d":5,"days_since_maintenance":220,"asset_criticality":5}'
```

6. Resultado y limitaciones

La API y seis pruebas automatizadas pasan en el entorno de ejecución utilizado. El modelo se entrenó correctamente.



7. Conclusión

El prototipo demuestra una ruta técnica reproducible para contenerizar y exponer un modelo de scoring mediante API y frontend. Cumple la validación funcional automatizada disponible y define un procedimiento de despliegue PaaS

Manual de despliegue en la nube

Contenerización, ejecución local y publicación PaaS mediante Amazon ECS Express Mode

1. Propósito y alcance

Este manual describe cómo ejecutar y publicar el prototipo de scoring de riesgo de falla. El proyecto continúa el business case de Fase 1: combina señales de condición, historial de mantenimiento y criticidad para priorizar alertas. La IA no genera órdenes automáticamente; la decisión permanece en el personal responsable.

Las pruebas usan exclusivamente datos sintéticos. No constituyen validación operacional en plantas ni medición de reducción de fallas.

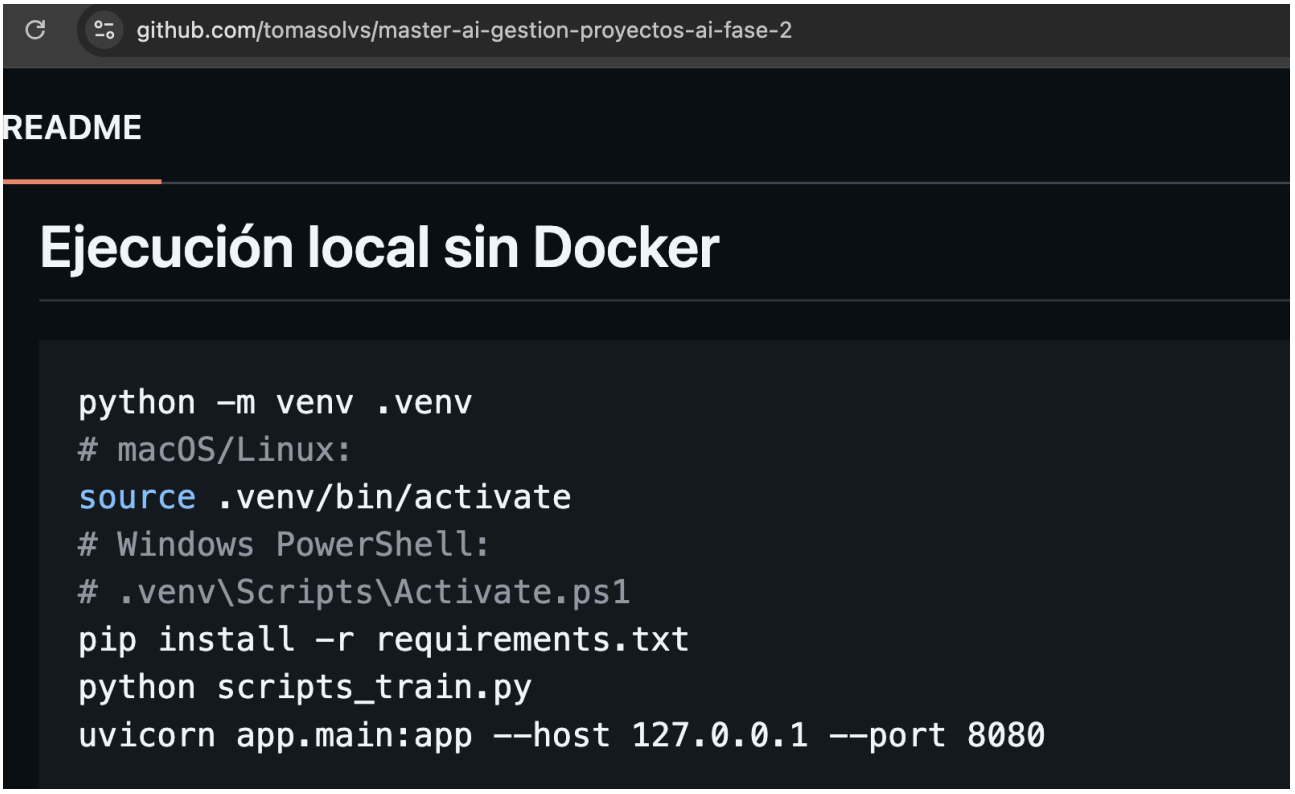
2. Arquitectura

Frontend web HTML/CSS/JavaScript → API FastAPI → pipeline Scikit-learn (Random Forest) → probabilidad experimental, nivel de riesgo y señales explicativas. Endpoints: GET /, GET /health, GET /model/metrics, POST /predict y documentación automática GET /docs.

3. Requisitos técnicos

- Python 3.11 para ejecución local; Docker Desktop o Docker Engine para contenerización.
- Docker CLI; puerto 8080 disponible; conexión a internet para instalar dependencias.
- Para AWS: cuenta AWS, permisos IAM para ECR y Amazon ECS, AWS CLI configurada y repositorio GitHub.
- Recursos de laboratorio recomendados: 1 vCPU y 2 GB RAM como punto de partida; ajustar tras medir consumo. No es dimensionamiento productivo.

4. Ejecución local sin Docker



- Crear entorno: python -m venv .venv
- Activar: source .venv/bin/activate (macOS/Linux) o .venv\\Scripts\\Activate.ps1 (PowerShell).
- Instalar: pip install -r requirements.txt
- Entrenar/reproducir artefacto: python scripts_train.py
- Iniciar: uvicorn app.main:app --host 127.0.0.1 --port 8080
- Abrir http://127.0.0.1:8080

5. Construcción y ejecución Docker

Desde la raíz del repositorio:

```
docker build -t bimbo-maintenance-ai:1.0 .

docker run --rm -p 8080:8080 --name maintenance-ai bimbo-maintenance-ai:1.0

curl http://localhost:8080/health
```

Para detener el contenedor: Ctrl+C en la terminal o docker stop maintenance-ai en otra terminal si se ejecutó en segundo plano.

6. Publicación PaaS en AWS

Estrategia recomendada: imagen Docker → Amazon Elastic Container Registry (ECR) → Amazon ECS Express Mode. AWS ha anunciado el cierre de App Runner para nuevos clientes y

recomienda ECS Express Mode como alternativa administrada. ECS Express Mode aprovisiona servicio ECS sobre Fargate, balanceador, escalamiento y red.

6.1 Preparar el repositorio GitHub

- Crear un repositorio privado o público vacío en GitHub (por ejemplo, bimbo-maintenance-ai).
- Descomprimir el proyecto y ejecutar git init, git add ., git commit -m 'Fase 2: prototipo de mantenimiento predictivo'.
- Agregar el remoto del repositorio propio y ejecutar git push -u origin main.
- No subir credenciales, claves, archivos .env ni datos reales. El .gitignore debe excluir entornos virtuales y secretos.

6.2 Construir y publicar imagen en ECR

```
aws configure   # configurar credenciales y región; nunca guardar claves en el repositorio

aws ecr create-repository --repository-name bimbo-maintenance-ai --region <REGION>

aws ecr get-login-password --region <REGION> | docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com

docker build -t bimbo-maintenance-ai:1.0 .

docker tag bimbo-maintenance-ai:1.0 <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/bimbo-maintenance-ai:1.0

docker push <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/bimbo-maintenance-ai:1.0
```

Sustituir <REGION> y <ACCOUNT_ID> por los valores reales de la cuenta. El rol/usuario debe contar con permisos mínimos para ECR. No compartir tokens ni credenciales.

6.3 Crear servicio ECS Express Mode

- En AWS Console, abrir Amazon ECS y elegir la opción para crear un servicio Express Mode (la disponibilidad de la experiencia de consola puede variar por región/cuenta).
- Seleccionar la imagen de ECR publicada y especificar el puerto del contenedor: 8080.
- Configurar los roles IAM requeridos: rol de ejecución de tareas y rol de infraestructura de Express Mode. Aplicar mínimo privilegio.
- Configurar health check path /health; iniciar con mínimo 1 tarea y máximo 2 para el ejercicio, sujeto a cuotas y costos.
- Crear el servicio y esperar el estado estable. Copiar el endpoint generado.
- Verificar GET /health, abrir /docs y realizar una predicción de prueba. Revisar logs y métricas en CloudWatch.
- Para actualizar: construir una nueva etiqueta de imagen, subirla a ECR y actualizar el servicio ECS a esa imagen. Probar health check y endpoint antes de considerar la versión validada.

La creación de recursos cloud puede generar cargos. Revisar precios vigentes de Fargate, balanceador, ECR, transferencia y CloudWatch antes de desplegar; eliminar los recursos al terminar el laboratorio.

7. Configuración y seguridad

- Escuchar en 0.0.0.0:8080 dentro del contenedor.
- No incluir credenciales ni datos personales en la imagen.
- Restringir permisos IAM, habilitar HTTPS en el endpoint administrado y limitar acceso administrativo.
- Activar logs; establecer retención; no registrar datos sensibles.
- No conectar redes OT/IT ni CMMS reales en este prototipo.
- Mantener revisión humana y procedimiento manual equivalente; el modelo no debe ser punto único de falla.

8. Herramientas de documentación

La documentación del repositorio se estructuró con apoyo de IA generativa y revisión técnica. El uso de Copilot y ChatGPT puede acelerar README, documentación general, runbooks y listas de verificación, pero las instrucciones, comandos, permisos, arquitectura y resultados deben verificarse contra el entorno real.